McMaster University

Lab 8

ELECENG 3EY4

Alaa Abdelsalam – abdela74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

March 28, 2024

Table of Contents

Contribution

Activities 1 to 19: Alaa, Tina, Marisa

Activity 1

The assumption under consideration pertains to the potential disregard of minor positional discrepancies between the "laser" frame, which denotes the LiDAR sensor's position, and the "base_link" frame, which serves as the central reference point of the vehicle. This simplification is predicated on the vertical alignment of the laser's mounting directly above the "base_link" along the z-axis, resulting in closely aligned x and y coordinates. The marginal displacement between the frames is deemed inconsequential and improbable to exert a significant influence on the force analysis within the x and y planes. Consequently, an obstacle detected by the LiDAR effectively obstructs the path of both the laser and the "base_link" frames, signifying overlapping detection of potential collision threats across both reference frames.

The angle, αi, denoting the deviation from the vehicle's frontal orientation to the obstacle, can be directly derived from the LiDAR scan data. As the zero angle of the "laser" frame aligns with the vehicle's x-axis, the angle information obtained from the LiDAR scan corresponds to the angle between the obstacle and the vehicle's forward direction. By applying the cosine law or basic trigonometric principles to the triangle formed by the base_link, laser, and obstacle (with the laser-to-obstacle line serving as the hypotenuse), αi can be computed. This calculation is crucial for understanding the obstacle's positioning in relation to the vehicle's trajectory and is imperative for making the necessary navigational adjustments to avoid collisions.

Activity 2

Derivation of $\delta_o^{k+1} = \operatorname{atan}\left(\frac{u_0 - \tan\delta_{joy}^{k+1}}{1 + u_0 \tan\delta_{joy}^{k+1}}\right)$:

Using:

$$\eta_\delta \frac{v_s^2}{l} \tan\delta_{joy}^{k+1} + (1-\eta_\delta) f_{oy} = \frac{v_s^2}{l} \tan\left(\delta_{joy}^{k+1} + \delta_o^{k+1}\right)$$

$$u_0 = \eta_\delta \tan\delta_{joy}^{k+1} + (1-\eta_\delta) \frac{l}{(\max(v_s^2, \epsilon))} f_{oy}$$

$\epsilon > 0$

Rearrange the Above Equations and Solve:

$$\eta_\delta \frac{v_s^2}{l} \tan \delta_{joy}^{k+1} + (1-\eta_\delta) f_{oy} = \frac{v_s^2}{l} \tan\left(\delta_{joy}^{k+1} + \delta_o^{k+1}\right)$$

$$\frac{l}{v_s^2}\left[\eta_\delta \frac{v_s^2}{l} \tan \delta_{joy}^{k+1} + (1-\eta_\delta) f_{oy}\right] = \frac{v_s^2}{l} \tan\left(\delta_{joy}^{k+1} + \delta_o^{k+1}\right)$$

$$\eta_\delta \tan \delta_{joy}^{k+1} + (1-\eta_\delta) f_{oy} = \tan\left(\delta_{joy}^{k+1} + \delta_o^{k+1}\right)$$

$$\tan(a+b) = \frac{\tan(a) + \tan(b)}{1 - \tan(a)\tan(b)}$$

$$u_0 = \eta_\delta \tan \delta_{joy}^{k+1} + (1-\eta_\delta) \frac{1}{v_s^2} f_{oy} = \frac{\tan\left(\delta_{joy}^{k+1}\right) + \tan(\delta_o^{k+1})}{1 - \tan\left(\delta_{joy}^{k+1}\right)\tan(\delta_o^{k+1})}$$

$$\tan\left(\delta_{joy}^{k+1}\right) + \tan(\delta_o^{k+1}) = u_0 - u_0 \tan\left(\delta_{joy}^{k+1}\right) + \tan(\delta_o^{k+1})$$

$$u_0 \tan\left(\delta_{joy}^{k+1}\right) + \tan(\delta_o^{k+1}) + \tan(\delta_o^{k+1}) = u_0 - \tan\left(\delta_{joy}^{k+1}\right)$$

$$\tan\left(\delta_{joy}^{k+1}\right)\left[1 + u_0 \tan\left(\delta_{joy}^{k+1}\right)\right] = u_0 - \tan\left(\delta_{joy}^{k+1}\right)$$

$$\delta_o^{k+1} = \operatorname{atan}\left(\frac{u_0 - \tan \delta_{joy}^{k+1}}{1 + u_0 \tan \delta_{joy}^{k+1}}\right)$$

Therefore, $\delta_o^{k+1} = \operatorname{atan}\left(\frac{u_0 - \tan \delta_{joy}^{k+1}}{1 + u_0 \tan \delta_{joy}^{k+1}}\right)$.

The design parameter $0 \leq \eta_\delta \leq 1$ has a significant role. Setting a parameter value within the range of 0 to 1 for η_δ enables the establishment of an equilibrium between the user's input and the collision avoidance algorithm. The adjustment of this parameter facilitates the fine-tuning of the collision avoidance system's impact on steering correction in relation to the driver's input, thereby establishing a desired response in terms of both safety and user experience. In the scenario where η_δ is equal to 1, the user's command supersedes the collision avoidance action entirely. Consequently, the corrective steering angle is determined exclusively by the driver's steering command, with the collision avoidance algorithm exerting no influence on the steering correction, effectively rendering the collision avoidance system inactive. In the scenario where η_δ is equal to 0, it signifies a condition in which the subsequent velocity command of the vehicle is exclusively determined by the corrective action of the collision avoidance algorithm. Under these circumstances, the corrective steering angle is entirely governed by the collision avoidance algorithm, rendering the driver's input inconsequential to the steering correction.

Activity 3

In activity 3, we copied the content of the “collision_assistance.py” file in the “node” directory of the “f1tenth_simulator” software stack. Then, we made this file executable by running the “chmod +x collision_assistance.py” command at the terminal. This can be seen in Figure 1 below.

Figure 1. Running “chmod +x collision_assistance.py” Command at the Terminal

Activity 4

In activity 4, we completed the missing code in the “collision_assistance.py” file for the function that finds potentially dangerous obstacles and their distance to the vehicle. This can be seen in Figure 2 below.

Completed Lines of Code in “collision_assistance.py” File:

index[1]=i+1

arg_ls_obs[i]=np.argmin(ls_ranges[self.obs_indx[i,0]: self.obs_indx[i,1],0])

arg_ls_obs[i]=self.obs_indx[i,0]+arg_ls_obs[i]

```
# Find unsafe obstacles and return number of unsafe obstacles
#  and the closest point of each unsafe obstacle to the vehicle
def obst_idnt(self,ls_ranges):

    indx=np.zeros((2),dtype=int)
    j=0
    nm_obs=0

    for i in range(self.ls_len_mod2):
        if ls_ranges[i,0]<=self.d_obs and i<self.ls_len_mod2-1:
            if j==0:
                indx[0]=i
                j=1
            indx[1]=i+1 #
        else:
            j=0
            if indx[1]-indx[0]>0:
                self.obs_indx[nm_obs,:]=indx
                nm_obs=nm_obs+1
                indx[1]=0;indx[0]=0

    arg_ls_obs=np.zeros(nm_obs,dtype=int)

    for i in range(nm_obs):
        arg_ls_obs[i]=np.argmin(ls_ranges[self.obs_indx[i,0]: self.obs_indx[i,1],0]) #
        arg_ls_obs[i]=self.obs_indx[i,0]+arg_ls_obs[i] #

    return nm_obs, arg_ls_obs
```

Figure 2. Completed Code in "collision_assistance.py" File

Activity 5

In activity 5, we completed the missing code in the "collision_assistance.py" file for the function "ptn_fld," which computes the total obstacles-related corrective forces acting on the vehicle. This can be seen in Figure 3 below.

Completed Lines of Code in "collision_assistance.py" File:

f_rep_x=f_rep_x+self.f_gain*(1-self.d_obs/d)*math.cos(theta)

f_rep_y=f_rep_y+self.f_gain*(1-self.d_obs/d)*math.sin(theta)

```
# compute corrective force acting on the vehicle
def ptn_fld(self, nm_obs,arg_ls_obs,ls_ranges):

    f_rep_x=0;f_rep_y=0

    for i in range(nm_obs):
        d=ls_ranges[arg_ls_obs[i],0]
        theta=ls_ranges[arg_ls_obs[i],1]

        f_rep_x=f_rep_x+self.f_gain*(1-self.d_obs/d)*math.cos(theta) #
        f_rep_y=f_rep_y+self.f_gain*(1-self.d_obs/d)*math.sin(theta) #

    return f_rep_x, f_rep_y
```

Figure 3. Completed Code in "collision_assistance.py" File

Activity 6

In activity 6, we completed the missing code in the "collision_assistance.py" file for the "lidar_callback" function. This can be seen in Figure 4 below.

Completed Lines of Code in "collision_assistance.py" File:

vel_d=self.etha_vel * self.vel_joy + (1-self.etha_vel) * self.vel

u0=self.etha_delta*math.tan(self.steer_joy)+(1-self.etha_vel) * f_tot_y * self.wheelbase/max(abs(self.vel),0.001) ** 2

The purpose of this part of the code is to control the vehicle's motion using desired velocity, "vel_d," and desired steering angle, "delta_d." In Figure 4, the first line calculates the desired velocity for the vehicle using a weighted sum of two components. "self.etha_vel" is a weighting factor to determine joystick velocity "self.vel_joy" on the "vel_d". The code adjusts for a reverse motion of the vehicle in the "if" statement that checks if the joystick velocity is positive and if "vel_d" is negative it moves backward. To calculate the desired steering angle "delta_d," the code determines a variable "u0" based on joystick inputs, current velocity, and other parameters. It then calculates "delta_d" using a mathematical expression involving the inverse tangent function "math.atan." these calculations determine the vehicle's steer angle controlled by the "self.steer_joy" joystick input.

```
vel_d= self.etha_vel * self.vel_joy + (1-self.etha_vel) * self.vel #

if self.vel_joy >=0 and vel_d < 0:
    vel_d = 0

u0=self.etha_delta*math.tan(self.steer_joy)+ (1-self.etha_vel) * f_tot_y * self.wheelbase/max(abs(self.vel),0.001) ** 2 #
delta_d=math.atan((u0-math.tan(self.steer_joy))/(1+u0*math.tan(self.steer_joy)))+self.steer_joy
```

Figure 4. Completed Code in "collision_assistance.py" File

Activity 7

In activity 7, we replaced the "behavior_controller.cpp," "mux.cpp," "simulator.launch," "experiment.launch," and "params.yaml" files in our existing software stack with the ones provided on Avenue for Lab 8. Then, we edited the values for the Ackerman-to-VESC command parameters in the "params.yaml" file based on the results of the calibration experiment from Lab 6. To ensure our calibration experiment results were accurate, we reconducted the calibration experiment and performed the necessary calculations, which were then updated in the "params.yaml" file. This can be seen in Figure 5 below. Then, we rebuilt the simulator package by using the "catkin_make" command.

Calibration Experiment Calculations:

$Speed\ to\ rpm\ Gain = 3182.18\ (default\ value)$

$Speed\ to\ rpm\ Offset = 0\ (default\ value)$

$Steering\ Angle\ to\ Servo\ Offset = 0.50\ (default\ value)$

Finding the Actual Steering Angle, δ:

Using:
$Length\ of\ McMaster\ AEV, l = 0.3m$

$Diameter, 2r = 1.25m$

$Radius, r = 0.625m$

$\delta = \tan^{-1}\left(\frac{l}{r}\right)$

$\delta = \tan^{-1}\left(\frac{0.3m}{0.625m}\right)$

$\delta = 0.448$

Finding the Steering Angle to Servo Gain, k_δ:

Using:

$Actual\ Steering\ Angle, \delta = 0.448$

$Desired\ Steering\ Angle\ from\ "params.yaml"\ File, \delta_d = 0.4189$

$Current\ Value\ of\ Gain\ from\ "params.yaml"\ File, \bar{k}_\delta = -0.88$

$$k_\delta = \frac{\delta_d}{\delta}\bar{k}_\delta$$

$$k_\delta = \frac{0.4189}{0.448}(-0.88)$$

$$k_\delta = -0.823$$

```
# (change these values based on your own calibration)

speed_to_erpm_gain: 3182.18
speed_to_erpm_offset: 0.0
steering_angle_to_servo_gain: -0.823
steering_angle_to_servo_offset: 0.50

driver_smoother_rate: 75.0 # messages/sec
```

Figure 5. Values Updated Based on Calibration Experiment

Activity 8

In activity 8, we examined the performance of the collision assistance avoidance algorithm in the simulation environment. To end this, we opened the "simulator.launch" file and ensured that the following lines were not commented out:

<node pkg="f1tenth_simulator" name="collision_assistance" type="collision_assistance.py" output="screen">
    <rosparam command="load" file="$(find f1tenth_simulator)/params.yaml"/>
</node>

Then, we opened the "params.yaml" file and took note of the following parameters related to the collision avoidance assistance algorithm:

collision_assistance_button_idx: 2 # X button
distance_to_obstacle_th: 1.0
force_gain: 8.0
velocity_correction_gain: 0.8
steering_correction_gain: 0.6

These parameters are shown in Figure 6 below. The "collision_assistance_button_idx" parameter assigns the button "X" on the joystick to activate the collision avoidance assistance algorithm. By pressing this button, the algorithm toggles "on" and "off." To activate full manual driving without collision assistance, "LB" on the joystick is used.

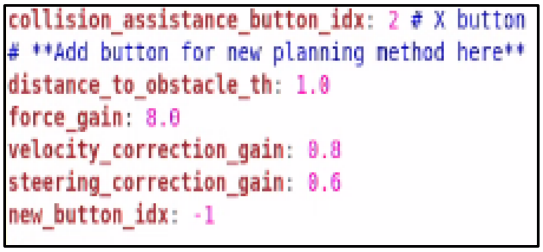

Figure 6. Parameters Related to the Collision Avoidance Assistance Algorithm in "params.yaml" File

Activity 9

In activity 9, we examined the code in the “collision_assistance.py” file and related the remaining parameters to the ones used in the collision avoidance assistance algorithm described in activity 8. By tuning these parameters, we are able to achieve the desired response from the vehicle in the simulation environment. To do this, we launched the simulator using the “roslaunch f1tenth_simulator simulator.launch” command. Then, we drove the vehicle in the virtual environment with and without collision avoidance assistance to notice any difference in its behavior. Using the simulation, we explored the impact of the parameters mentioned above on the vehicle response. We noted that when collision avoidance assistance is utilized and a collision occurs, the vehicle halts its movement and remains stationary even when attempting to use the joystick. Conversely, in scenarios where collision avoidance assistance was not employed and a collision took place, the vehicle continued moving after the collision when joystick inputs were applied. Activation of the collision avoidance assistance feature is initiated by pressing the “X” and “LB” buttons on the joystick. After activation, if the car approaches a wall, the program detects an imminent collision and promptly disables the joystick to prevent any additional control inputs that might result in a collision. This action effectively brings the car to a halt, mitigating the risk of potential damage. Furthermore, activating collision assistance with the “X” button and subsequently pressing “RB” enables autonomous navigation, empowering the car to autonomously navigate the track while avoiding collisions with walls or obstacles. Conversely, pressing “LB” switches the system to full manual mode, deactivating collision assistance. This manual override was demonstrated in the first image, where the absence of collision assistance resulted in the car colliding with a wall.

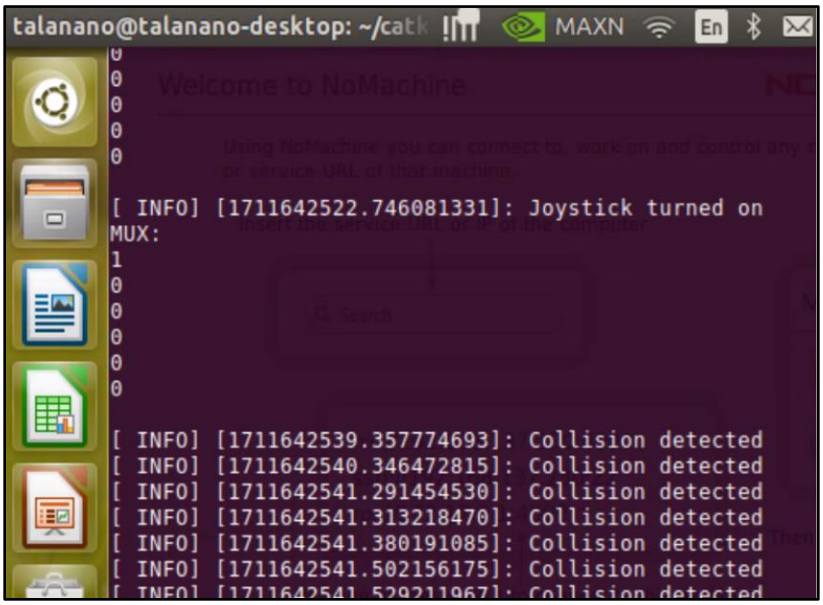

Figure 7. Observation of Collision Detected

Activity 10

In activity 10, before utilizing the LiDAR for collision avoidance, we first ensured that we were using the right value for the parameter called “scan_beams” in the “params.yaml” file. This value denotes the number of scan beams in one full LiDAR rotation. The LiDAR angular resolution is given by $2\pi/scan\_beams$. We copied the following lines from the “experiment.launch” file into a new launch file so we can run the RPLiDAR by itself in “BOOST” (high resolution) mode:

```
<!-- launch rplidar driver node-->
<node pkg="rplidar_ros" name="rplidarNode" type="rplidarNode">
<param name="port" value="/dev/sensors/rplidar" />
<param name="frame_id" type="string" value="laser"/>
<param name="inverted" type="bool" value="false"/>
<param name="angle_compensate" type="bool" value="true"/>
<param name="scan_mode" type="string" value="Boost"/>
</node>
```

Then, we ran the new launch file and observed the value of the data field “angle_increment” in the “scan” topic by using the “rostopic echo /scan/angle_increment” command. This value is seen in Figure 8 below.

Figure 8. Value of Data Field “angle_increment”

This value is the actual LiDAR angular resolution. Using this value, we calculated the correct value for the “scan_beams” parameter. Then, we updated this value in the “params.yaml” file.

Calculation of scan_beams Value:

$$LiDAR\ Anuglar\ Resolution = \frac{2\pi}{scan\_beams}$$

$$scan\_beams = \frac{2\pi}{LiDAR\ Anuglar\ Resolution}$$

$$scan\_beams = \frac{2\pi}{angle\_increment}$$

$$scan\_beams = \frac{2\pi}{0.00871450919658}$$

$$scan\_beams = 721$$

Therefore, the correct value for the “scan_beams” parameter is 721.

Activity 11

In activity 11, we examined the vehicle behavior in the experiment. To do this, we opened the “experiment.launch” file and ensured that the collision assistance avoidance node was launched. Then, we ran the launch file using the “roslaunch f1tenth_simulator experiment.launch” command. Then, we drove the vehicle while having collision avoidance assistance activated. Following this, we evaluated its response. We fine-tuned the parameters of the algorithm and observed the impact of these parameters on the vehicle response. We observed that when collision avoidance is activated, it detects immediate collisions and automatically tilts the wheels to avoid collision. To resolve the issue of the vehicle moving in the reverse direction, we changed the “speed_to_erpm_gain” to negative 3182.18. We reduced the “vehicle_velocity” to 0.5 because the vehicle was very fast and not able to avoid obstacles in time.

Activity 12

Derivation of $\ddot{d}_{lr}$:

Using $\ddot{d}_r = \frac{v_s}{l}\cos\alpha_r \tan\delta$ and $\ddot{d}_l = -\frac{v_s}{l}\cos\alpha_l \tan\delta$:

$$\ddot{d}_{lr} = \ddot{d}_l - \ddot{d}_r$$

$$\ddot{d}_{lr} = -\frac{v_s}{l}\cos\alpha_l \tan\delta - \frac{v_s}{l}\cos\alpha_r \tan\delta$$

$$\ddot{d}_{lr} = -\frac{v_s}{l}(\cos\alpha_l \tan\delta + \cos\alpha_r \tan\delta)$$

$$\ddot{d}_{lr} = -\frac{v_s^2}{l}(\cos\alpha_l + \cos\alpha_r)\tan\delta$$

Therefore, $\ddot{d}_{lr} = -\frac{v_s^2}{l}(\cos\alpha_l + \cos\alpha_r)\tan\delta$.

Derivation of δ_{lr}:

Using $\delta_r = \text{atan}\left(\frac{l}{v_s^2\cos\alpha_r}(-k_p\tilde{d}_r - k_d\dot{d}_r)\right)$ and $\delta_l = \text{atan}\left(\frac{l}{v_s^2\cos\alpha_l}(-k_p\tilde{d}_l - k_d\dot{d}_l)\right)$:

$$\delta_{lr} = \delta_l - \delta_r$$

$$\delta_{lr} = \text{atan}\left(\frac{l}{v_s^2\cos\alpha_l}(-k_p\tilde{d}_l - k_d\dot{d}_l)\right) - \text{atan}\left(\frac{l}{v_s^2\cos\alpha_r}(-k_p\tilde{d}_r - k_d\dot{d}_r)\right)$$

$$\delta_{lr} = \text{atan}\left(\frac{l}{v_s^2\cos\alpha_l}(-k_p\tilde{d}_l - k_d\dot{d}_l) - \frac{l}{v_s^2\cos\alpha_r}(-k_p\tilde{d}_r - k_d\dot{d}_r)\right)$$

$$\delta_{lr} = \text{atan}\left(\frac{l(\cos\alpha_r - \cos\alpha_l)}{v_s^2\cos\alpha_l\cos\alpha_r}(-k_p\tilde{d}_l - k_d\dot{d}_l) + (-k_p\tilde{d}_r - k_d\dot{d}_r)\right)$$

$$\delta_{lr} = \text{atan}\left(\frac{l(\cos\alpha_r - \cos\alpha_l)}{v_s^2\cos\alpha_l\cos\alpha_r}(-k_p\tilde{d}_l - k_d\dot{d}_l - k_p\tilde{d}_r - k_d\dot{d}_r)\right)$$

$$\delta_{lr} = \text{atan}\left(\frac{l(\cos\alpha_r - \cos\alpha_l)}{v_s^2\cos\alpha_l\cos\alpha_r}(-k_p(\tilde{d}_l + \tilde{d}_r) - k_d(\dot{d}_l + \dot{d}_r))\right)$$

$$\delta_{lr} = \text{atan}\left(\frac{-l}{v_s^2(\cos\alpha_l + \cos\alpha_r)}(-k_p\tilde{d}_{lr} - k_d\dot{d}_{lr})\right)$$

Therefore, $\delta_{lr} = \text{atan}\left(\frac{-l}{v_s^2(\cos\alpha_l+\cos\alpha_r)}(-k_p\tilde{d}_{lr} - k_d\dot{d}_{lr})\right)$.

Derivation of $H_{cl}(s)$:

$$\ddot{d}_{lr} + k_d\dot{d}_{lr} + k_p d_{lr} = k_p d_{ir}^{des}$$

$$S^2 D(s) + k_d S\, D(s) + k_p D(s) = k_p D_{ir}^{des}$$

$$H_{cl}(s) = \frac{D(s)}{D_{lr}^{des}(s)}$$

$$H_{cl}(s) = \frac{k_p}{s^2 + k_d s + k_p}$$

Therefore, $H_{cl}(s) = \frac{k_p}{s^2+k_d s+k_p}$.

Derivation of α_l (from page 14 in Lab 8 Manual):

$$\alpha_l = -\beta_l + \frac{3\pi}{2} - \angle b_l$$

Derivation of d_r (from page 14 in Lab 8 Manual):

$$d_r = dis\_lsr\_br * \cos(betar)$$

Derivation of d_l (from page 14 in Lab 8 Manual):

$$d_r = dis\_lsr\_bl * \cos(beta1)$$

Activity 13

Calculation of Steady-State Tracking Error in Response to a Constant Distance Command in the Closed-Loop Control:

Using the Final Value Theorem:

$$e_{ss} = \lim_{s \to 0} s \left(D_{lr}^{des}(s) - \frac{1}{H_{cl}(s) D_{lr}^{des}(s)} \right)$$

$$e_{ss} = \lim_{s \to 0} s \left(\frac{d_{lr}^{des}(s)}{s} - \frac{s^2 + k_d s + k_p}{k_p \frac{d_{lr}^{des}(s)}{s}} \right)$$

$$e_{ss} = d_{lr}^{des} - \frac{k_p}{k_d * d_{lr}^{des}}$$

$$e_{ss} = 0$$

Therefore, the steady-state tracking error in response to the constant distance command in the closed-loop control is equal to 0. This indicates that assuming the system is stable, it will be able to perfectly track the desired constant distance in steady-state.

Activity 14

In activity 14, we copied the new map files called “env_map.pgm” and “env_map.yaml,” which were in the “/maps” directory of the F1TENTH_SIMULATOR package and ensured that the “simulator.launch” file was modified to use this map in our simulations.

We copied the contents of the “navigation_lab8_inc.py” file from Avenue into the “navigation.py” file in that “node” subdirectory of the F1TENTH_SIMULATOR package. Then, we added the following lines to both the “experiment.launch” and “simulator.launch” files of our package:

<node pkg="f1tenth_simulator" name="navigation" type="navigation.py" output="screen">
<rosparam command="load" file="$ (find f1tenth_simulator)/params.yaml"/>
</node>

Activity 15

In activity 15, we completed the code to implement the algorithm in activity 14. This can be seen in Figures 9, 10, and 11 below.

Completed Lines of Code in “navigation.py” File:

self.thetal=self.angle_br - self.angle_al
self.thetar=self.angle_ar - self.angle_br

betal=math.atan(dis_lsr_al*math.cos(self.thetal) – (dis_lsr_bl)/(dis_lsr_al*math.sin(self.thetal))
betar= math.atan(dis_lsr_ar*math.cos(self.thetar) – (dis_lsr_br)/(dis_lsr_ar*math.sin(self.thetar))
alphal=-betal - self.angle_bl +3*math.pi/2
alphar=betar - self.angle_br + math.pi/2

d_tilde=dl-dr-self.CenterOffset
d_tilde_dot=-self.vel*math.sin(alphal)-self.vel*math.sin(alphar)
delta_d=
math.atan((self.wheelbase*(self.k_p*d_tilde+self.k_d*d_tilde_dot))/((self.vel**2)*(math.cos(alphal)+math.cos(alphar))))

```
  self.vel = 0

  self.thetal= self.angle_bl - self.angle_al   # added
  self.thetar= self.angle_ar - self.angle_br   # added
```

Figure 9. Completed Code in “navigation.py” File

```
        betal= math.atan(dis_lsr_al*math.cos(self.thetal) -
dis_lsr_bl)/(dis_lsr_al*math.sin(self.thetal)) # added

        betar= math.atan(dis_lsr_ar*math.cos(self.thetar) -
dis_lsr_br)/(dis_lsr_ar*math.sin(self.thetar)) # added

        alphal = -betal - self.angle_bl + 3*math.pi/2 # added
        alphar = betar - self.angle_br + math.pi/2 # added
```

Figure 10. Completed Code in “navigation.py” File

```
            else :
                d_tilde= dl-dr-self.CenterOffset # added
                d_tilde_dot= -self.vel*math.sin(alphal)-
self.vel*math.sin(alphar)  # added
                delta_d =
math.atan((self.wheelbase*(self.k_p*d_tilde+self.k_d*d_tilde_dot
))/((self.vel**2)*(math.cos(alphal)+math.cos(alphar)))) # added
```

Figure 11. Completed Code in "navigation.py" File

Activity 16

In activity 16, this code adjusts the velocity of the vehicle based on real-time data from the LiDAR sensor to ensure that the vehicle slows down or stops when detecting an obstacle that is within a certain range.

"min_distance = min(data.ranges[-sec_len+int(self.scan_beams/2):sec_len+int(self.scan_beams/2)])":

This line calculates the minimum distance from the LiDAR data, where "data.ranges" represent the range data obtained from the LiDAR sensor. The rest of the code is an operation that extracts a subset of range data from "data.ranges." The function "min" finds the minimum value within the selected range of data.

"velocity_scale = 1-math.exp(-max(min_distance-self.stop_distance,0)/self.stop_distance_decay)":

This line calculates a velocity scaling factor based on the minimum distance obtained from the sensor. If the difference between the minimum distance and the stop distance threshold is negative, its replaced with zero to ensure the scaling factor doesn't become negative. The "self.stop_distance_decay" is a decay factor that controls how fast the velocity scales down as the minimum distance decreases. The "math.exp()" calculates the exponential of the given value. Then subtract the result from 1 to ensure that the velocity scaling factor decreases as the minimum distance decreases (1-).

"velocity=velocity_scale*self.vehicle_velocity":

This line calculates the scaled velocity based on the velocity scaling factor obtained in the previous step. It simply multiplies the scaling factor "velocity_scale" by the base vehicle velocity "self.vehicle_velocity." This scaled velocity adjusts the speed of the vehicle based on the obstacle detected by the sensor.

Activity 17

In activity 17, we controlled the vehicle using three different modes including “distance-to-left wall,” “distance-to-right wall,” and an offset from the center position of the walls. By setting the value of the “TrackWall” parameter in the “params.yaml” file, we were able to choose the driving mode.

Using the “distance-to-left wall” mode, we observed the vehicle's behaviour in the simulation environment. In this operational mode, the vehicle consistently maintains a set distance from the left wall. This mode proves advantageous in scenarios where the left wall offers greater reliability or presents fewer obstructions. However, it is important to note that this approach may result in the vehicle deviating from the center of the hallway, potentially increasing the risk of collisions.

Using the “distance-to-right wall” mode, we observed the vehicle's behaviour in the simulation environment. In this operational mode, the vehicle consistently maintains a fixed distance from the right wall. This feature proves advantageous in scenarios where the right wall is deemed more dependable or presents fewer obstructions. Analogous to the preceding control mechanism, a potential drawback is the vehicle's potential deviation from the hallway center, thereby increasing the risk of collisions.

Using the mode where we applied an offset from the center position of the walls, we observed the vehicle's behaviour in the simulation environment. In this operational mode, the vehicle maintains a central position within the hallway, ensuring equal proximity to both the left and right walls. Despite this, this approach may not be optimally effective in hallways with uneven dimensions or where one wall exhibits greater reliability or contains differing obstacles compared to the other.

Each control mode presents distinct challenges. When utilizing the left or right wall for guiding the vehicle, potential difficulties arise if the vehicle strays too close to the opposite wall, particularly in hallways with varying configurations and obstacles. Similarly, in the center position control mode, achieving a balance in the distance between both walls proves to be a challenging task, especially in hallways with diverse configurations.

The k_p parameter is known as the controller gain, k_d and the parameter is known as the derivative gain. The controller gain parameter influences the speed and aggressiveness of the response to deviations from the desired distance. A higher value results in more rapid corrections, but excessively high values may lead to overshooting and oscillations. The derivative gain parameter introduces damping to the system, effectively mitigating overshoot and oscillations. By taking into account the rate of change of error, it contributes to stabilizing the overall response.

When determining the value for k_p, it is advisable to commence with a moderate setting and then progressively increase it until the system demonstrates prompt responsiveness, while avoiding excessive overshoot or oscillations. In the case of k_d, it is essential to fine-tune the value to attain optimal damping. Excessive damping may impede response time, while insufficient

damping can lead to undesirable oscillations. When determining the optimal values of both gains, a systematic approach involving trial and error can be employed to find the values that provide the desired response from the system.

Activity 18

In activity 18, we started the vehicle control system using the “roslaunch f1tenth_simulator experiment.launch” command. By toggling “RB” on the joystick, we are able to activate and deactivate the self-driving mode. By pressing “LB” on the joystick while in self-driving, we are able to deactivate self-driving and immediately switch to the manual driving mode. To ensure we do not lose communication between the joystick and the vehicle on-board computer, we remained within reach of the vehicle.

Activity 19

We observed a difference between the vehicle’s behaviour in the simulations and in the experiment. In the experimental scenario, the vehicle demonstrates accelerated movement, and with the collision assistance feature activated, it exhibits increased swerving and turning compared to its behavior in the simulated environment. The vehicle’s behavior notably included increased swerving and turning as compared to its performance in the simulation. Furthermore, the simulation environment featured a narrower hallway devoid of open doors or corridors, resulting in distinct vehicle responses such as swerving, turning, or maintaining its trajectory when encountering these elements. The vehicle behaves noticeably faster in the real world compared to the simulation, offering a more precise comprehension of how a car responds with collision assistance. The simulation exhibits significant glitches and inaccuracies in response times.